

Minimum Spanning Tree

Lardinois Joachim s2402075
Aleksandr Pavlov s2400691

15 décembre 2025

Table des matières

1	Introduction	1
2	Guide d'utilisation du programme	2
2.1	Mode commande classique	2
2.2	Mode interactif	2
2.3	Remarques pratiques	3
3	Trouver le MST d'un graphe	4
3.1	Algorithme de Kruskal	4
3.2	Algorithme de Prim	4
3.3	Algorithme par suppression inverse	4
4	Trouver une seconde solution	5
5	Mettre à jour le MST dynamiquement	6
5.1	Augmenter un poids ou retirer une arête	6
5.2	Diminuer un poids, ou rajouter une arête	6

1 Introduction

Pour le cours de théorie des graphes, nous (Alexandr Pavlov et Joachim Lardinois) avons du faire un programme capable de 3 choses. Le programme est capable de lire un fichier contenant un graphe, et d'écrire dans un autre fichier un sous-arbre couvrant de poids minimal (MST) de ce graphe. Ensuite, il doit pouvoir trouver un second MST différent du premier. Finalement, en modifiant un graphe, notre programme doit mettre à jour un MST de ce graphe, idéalement plus rapidement que en le reconstruisant complètement.

2 Guide d'utilisation du programme

Pour exécuter le programme, ouvrez un terminal dans le répertoire du projet et lancez :

```
python -m src.main
```

Le programme dispose de deux modes principaux : un mode commande classique et un mode interactif.

2.1 Mode commande classique

Pour obtenir l'aide générale et les options disponibles, utilisez :

```
python -m src.main -h
```

La sortie affiche :

```
usage: Graphoni [-h] [-a {kruskal,prim,reverse}] [-s] [-i] [-v] [input] [output]
```

Util for finding MST of graph

positional arguments:

input	input file
output	output file

options:

-h, --help	show this help message and exit
-a, --algorithm {kruskal,prim,reverse}	choose algorithm
-s, --second	find second MST
-i, --interactive	run interactive mode
-v, --version	show version

Explications des options :

- **-a, --algorithm** : choisir l'algorithme pour le calcul de l'arbre couvrant minimal (Kruskal, Prim ou Reverse Delete). Par défaut : Kruskal.
- **-s, --second** : calculer le second MST.
- **-i, --interactive** : lancer le mode interactif.
- **input** : fichier d'entrée contenant le graphe.
- **output** : fichier de sortie pour enregistrer le résultat.

2.2 Mode interactif

Le mode interactif permet de modifier le graphe et de calculer le MST en temps réel. Pour l'activer, lancez :

```
python -m src.main -i
```

Les principales commandes disponibles :

- **load** <path> : charger un graphe depuis un fichier.
- **save** <path> : enregistrer le graphe ou le MST dans un fichier.
- **add** <u> (<v> <weight>) : ajouter un sommet ou une arête au graphe.
- **remove** <u> (<v>) : supprimer un sommet ou une arête.
- **mst** (<algorithm> -s) : calculer le MST. On peut spécifier l'algorithme et l'option step-by-step.
- **update** <u> <v> (<weight>) : mettre à jour le graphe et recalculer le MST après modification.
- **second** : trouver le second MST.
- **keep** : remplacer le graphe original par le graphe résultat pour les prochaines opérations.
- **origin** : afficher le graphe original.
- **repos** : repositionner le graphe.
- **help** : afficher l'aide dans le mode interactif.
- **exit** : quitter le mode interactif.

2.3 Remarques pratiques

- Toujours lancer le programme depuis le répertoire principal du projet pour que les chemins relatifs fonctionnent correctement.
- Le mode interactif est particulièrement utile pour tester différentes modifications sur le graphe sans avoir à relancer le programme.

3 Trouver le MST d'un graphe

Un arbre couvrant minimal (MST) est un arbre couvrant dont le poids total est le plus petit possible.

Dans le cadre du projet, trois algorithmes classiques de construction d'un MST ont été réalisés : l'algorithme de Kruskal, l'algorithme de Prim et l'algorithme par suppression inverse.

3.1 Algorithme de Kruskal

Kruskal trie les arêtes par poids et les ajoute une à une à l'arbre, en évitant de former des cycles. La première version implémentait une vérification explicite des cycles, ce qui pour un graphe complet de 1000 sommets prenait environ une seconde. La version améliorée utilise une structure DSU (Union-Find), ce qui réduit le temps de calcul d'environ moitié. Cet algorithme s'est révélé très rapide par rapport aux autres méthodes.

3.2 Algorithme de Prim

Prim construit l'arbre à partir d'un sommet de départ : à chaque étape, on choisit l'arête de poids minimal qui relie l'arbre partiel à un sommet non encore inclus. Dans notre implémentation, cette version a montré des performances bien inférieures à Kruskal. Mais on peut l'améliorer considérablement en utilisant une structure de données différente pour stocker les arêtes.

3.3 Algorithme par suppression inverse

Cette méthode trie les arêtes par poids décroissant et tente de supprimer successivement l'arête la plus lourde tout en conservant la connexité. Elle s'est avérée plus lente comparée aux autres. On peut l'améliorer en améliorant la façon dont on vérifie que le graphe est connexe.

4 Trouver une seconde solution

Pour trouver un autre MST, il nous faut un MST, avec comme restriction que au moins une arrête doit être différente du premier MST.

Pour la prochaine question, nous avons du faire une fonction qui modifie un MST en fonction d'un changement sur un graphe. En faisant un peu de chipotage, on peut réutiliser cette fonction pour trouver un MST sur un même graphe, mais en empêchant une arrête de se retrouver dans le nouveau MST.

Une fois ça fait, on peut utiliser cette fonction sur notre MST, en retirant une arrête du MST donné. Si cela nous donne un meilleur résultat que précédemment trouvé, on enregistre ce résultat, et recommence avec une autre arrête.

Une fois que toutes les arrêtes sont passées, ou que l'on a trouvé un MST avec le même poids total que le premier MST, on peut s'arrêter et on a notre meilleur prochain candidat comme MST.

5 Mettre à jour le MST dynamiquement

Pour mettre le MST à jour dynamiquement, on peut séparer le problème en plusieurs cas.

Si on retire une arête du graphe (ou on augmente son poids), mais que celle-ci n'était pas dans le MST, celui-ci ne change pas. On peut donc garder le MST comme il était.

Si on diminue le poids d'une arête, cette fois-ci dans le MST, celui-ci reste à nouveau le même.

Par contre, il nous reste le cas de si on augmente le poids (ou qu'on retire) une arête du MST, et le cas de si on rajoute une arête ou diminue le poids d'une arête hors du MST.

5.1 Augmenter un poids ou retirer une arête

Supposons que le changement depuis le graphe précédent est que le poids d'une arête a augmenté.

Le MST étant un arbre, n'importe quelle arête sépare le graphe en 2 composantes. Ces 2 composantes sont aussi les MST des sous-graphes contenant les sommets de leurs composantes respectives. Ne changeant qu'une arête, celle-ci sépare le graphe en 2 sous-MST. La question devient donc, quelle arête doit-on utiliser pour reconnecter ces composantes ?

Si au lieu d'avoir augmenté le poids d'une arête, on en a retiré une, un même raisonnement nous amène à la même conclusion, avec pour seule différence le fait que l'arête retirée ne peut plus être choisie pour connecter le graphe.

Pour notre algorithme, nous allons retirer l'arête dans tout les cas, et ensuite la réajouter si nécessaire. Cela nous permet de ne considérer que le cas de retirer une arête.

Il faut aussi faire attention à si on retire une arête, il est possible que le graphe ne soit plus connexe.

5.2 Diminuer un poids, ou rajouter une arête

Supposons que le changement est qu'un poids a diminué (hors du MST), ou qu'une arête a été rajoutée.

Dans ce cas-ci, on doit choisir d'inclure ou non cette arête dans notre MST, et si oui, quelle arête retirer. Si on rajoute l'arête à notre MST, on rajoute 1 cycle dans le MST. Pour avoir à nouveau un MST, il faut donc "casser" ce cycle, que l'on peut faire en retirant n'importe quelle arête du cycle. Mais on veut un MST, donc on retire l'arête avec le poids le plus grand.

Références

- [1] Wikipedia. *Minimum spanning tree*. URL : https://en.wikipedia.org/wiki/Minimum_spanning_tree
- [2] Habr. *Minimum Spanning Tree Algorithms*. URL : <https://habr.com/ru/articles/569444/>
- [3] YouTube. *Minimum Spanning Tree Algorithms — Kruskal & Prim*. URL : <https://www.youtube.com/watch?v=Yk7L64gFuzk>
- [4] B. Korte, J. Vygen. *Combinatorial Optimization : Theory and Algorithms*. URL : <https://link.springer.com/book/10.1007/978-3-662-56039-6>.